A Comparative Study of Convolutional Neural Network Models for Rice Plant Leaf Disease Classification

Kshitij Ayush[1], Aditya Das[1], Bibek Kundu[1], Manomita Chakraborty[2], Subhash Barman[1], and Saroj Kumar Biswas[3]

[1]Jalpaiguri Government Engineering College, West Bengal, India
[2]School of Computer Science and Engineering, VIT-AP university, Andhra Pradesh, India
[3]National Institute of Technology Silchar, Assam, India

Abstract

Rice is a crucial staple food worldwide, but its production is significantly hampered by leaf diseases, leading to substantial loss of yield. Early and accurate disease detection is vital for effective crop management. Although traditional machine learning methods have been applied, they often rely on laborious handcrafted feature extraction. Deep Learning, particularly Convolutional Neural Networks (CNNs), offers a more robust solution by automatically learning features from images. This study presents a comprehensive comparative analysis of ten prominent pre-trained CNN architectures (including VGGNet, ResNet, DenseNet, and Xception) for classifying rice leaf diseases into four categories: Healthy, Bacterial Blight, Blast, and Brownspot. These models are evaluated in three distinct data pipelines: an original imbalanced dataset, a geometrically augmented dataset, and a balanced dataset augmented with synthetic images from a Wasserstein Generative Adversarial Network with Gradient Penalty (WGAN-GP). The findings reveal a significant generalization gap, with models achieving 99-100% validation accuracy but only 79-92% test accuracy. When trained on the balanced dataset, the ResNet101 architecture demonstrated superior performance, achieving the highest test accuracy of 92%. This result underscores the critical importance of robust data augmentation strategies in conjunction with a suitable model architecture, for developing effective and generalizable disease detection systems in agriculture.

Keywords: *Smart Farming, rice leaf diseases, image data analysis, Deep Learning, CNNs, WGAN-GP, generalizability*

1 Introduction

Rice (*Oryza sativa*) stands as a pivotal global crop, forming the cornerstone of sustenance and economic stability for a significant portion of the world's population [1]. As the primary staple food for more than half of humanity, it provides over 20% of the daily caloric intake for an estimated 3.5 billion people [2]. Historically and presently, rice cultivation has been integral to the economic development of numerous agrarian nations, particularly in Asia, Sub-Saharan Africa, and South America, where it serves as a primary source of income for millions of rural households [2]. However, the demand for this essential grain is intensifying in tandem with global population growth, which is projected to reach 9.7 billion by 2050 [2]. To meet this escalating demand and ensure global food security, it is estimated that an additional 116 million tons of rice will be needed by 2035. This imperative is severely undermined by compounding threats, including climate change and widespread pathogenic diseases such as Bacterial Blight, Blast, and Brown Spot [2]. These diseases are responsible for a staggeringly high 20% to 40% reduction in annual yield globally [2]. This figure represents more than just an agricultural loss; it constitutes a persistent threat to global food security, with the potential to trigger severe price shocks in importing nations and exacerbate food insecurity for the world's most vulnerable populations.

Historically, the identification of these diseases has relied on manual visual inspection by farmers or agricultural experts [5]. This traditional diagnostic process, while foundational, is fraught with fundamental limitations that render it inadequate for modern agriculture [6, 7]. It is inherently subjective and error-prone [6], contingent on human judgment that varies significantly between individuals and is susceptible to misinterpretation [1]. Diagnosis is particularly difficult when visual symptoms of different diseases, such as the circular lesions of Brown Spot and Leaf Smut, appear highly similar [1]. Furthermore, this process is labor-intensive and time-consuming, making it impractical for the systematic monitoring of extensive farm operations [1]. Accurate identification often necessitates the specialized knowledge of trained plant pathologists, but such expertise is a scarce resource and frequently inaccessible to farmers in remote regions [1, 2]. These constraints create a critical knowledge gap that leads to delayed detection, allowing infections to spread significantly before effective control measures can be implemented [2].

To overcome the limitations of manual inspection, early computational research introduced traditional machine learning (ML) algorithms such as Support Vector Machines (SVM) and Random Forests (RF) [7]. These methods typically followed a structured workflow of image preprocessing to enhance quality, followed by feature extraction and classification [8, 9]. While representing a significant step toward automation, this paradigm was constrained by a critical bottleneck: the reliance on handcrafted feature extraction [7]. Domain experts were required to manually design algorithms to extract specific features, such as color metrics from histograms, texture descriptors like the Gray-Level Co-occurrence Matrix (GLCM), or shape-based characteristics using Scale-Invariant Feature Transform (SIFT) [2]. Although these engineered features performed adequately under controlled conditions, they proved "brittle" and lacked generalizability [2]. Their effectiveness degraded significantly when faced with real-world complexities, such as variations in lighting, complex backgrounds, or the natural morphological diversity of disease symptoms [5]. The system's performance was fundamentally constrained by the programmer's ability to codify expertise into static rules, resulting in a model incapable of adapting to environmental variability [9].

The inherent limitations of handcrafted feature extraction catalyzed a paradigm shift toward Deep Learning, specifically Convolutional Neural Networks (CNNs) [11]. Unlike traditional ML, CNNs automatically learn discriminative features directly from raw images, moving the field from "feature engineering" to "feature learning" [2]. The superiority of CNNs lies in their architecture, which utilizes local connectivity and shared weights [1] to efficiently process spatial data. Through hierarchical feature learning, CNNs extract patterns of increasing complexity—from simple edges and textures in initial layers to complex, abstract lesion shapes in deeper layers—without human intervention [7]. Foundational architectures have since validated this approach: VGGNet demonstrated the importance of network depth [12]; ResNet introduced "skip connections" to enable the training of substantially deeper networks by mitigating the vanishing gradient problem [14, 15]; and EfficientNet optimized performance through principled compound scaling [13]. These models offer a data-driven form of "visual reasoning" capable of integrating local patterns into a global understanding of the image.

Motivated by these technological advancements, the present study performs a comprehensive comparative analysis of ten prominent pre-trained CNN architectures—including VGGNet, ResNet, DenseNet, and Xception—for classifying rice leaf diseases into four categories: Healthy, Bacterial Blight, Blast, and Brown Spot. The paper is organized as follows: Section 2 provides an overview of prior research work in disease detection of rice leaves using machine learning; Section 3 describes the details on the three pipelines that were used in this project, along the training and evaluation methodology; Section 4 deals with summarizing the results of the evaluation and includes some related discussion; finally Section 5 provides the concluding remarks and some discussion on the future of the project.

2 Literature Survey

Automated rice leaf disease detection has undergone a clear methodological evolution, progressing from traditional machine learning techniques to advanced deep learning frameworks. This transition has

been driven by the need for higher accuracy, reduced reliance on manual feature design, and improved generalization under real-world agricultural conditions. In this section, existing literature is reviewed following a logical progression: traditional machine learning approaches, early CNN-based methods, advanced CNN and attention-based architectures, and GAN-based data augmentation. The discussion explicitly highlights how the limitations of prior work motivate the design choices in the present study.

2.1 Traditional Machine Learning Methods

Initial research efforts in rice leaf disease classification primarily employed conventional machine learning (ML) algorithms such as Support Vector Machines (SVM), K-Nearest Neighbors (KNN), and Random Forests (RF). These approaches relied on a multi-stage pipeline consisting of image preprocessing, handcrafted feature extraction, and classification. Prajapati et al. [17], for example, used SVM classifiers with manually engineered color, shape, and texture features, achieving a test accuracy of 73.33%. Similar studies applied clustering techniques such as K-means for lesion segmentation followed by ML-based classification.

Although these methods established the feasibility of automated disease detection, their dependence on handcrafted features significantly constrained robustness and scalability. Feature engineering is labor-intensive, requires domain expertise, and struggles to capture the high intra-class variability caused by changes in lighting, background, disease progression, and image acquisition conditions [7, 10]. As a result, traditional ML models often perform well only under controlled conditions and fail to generalize across datasets. These limitations directly motivate the shift toward deep learning approaches in this study, which utilize end-to-end learning to capture discriminative representations directly from raw images.

2.2 Early CNN-Based Approaches

The adoption of Convolutional Neural Networks (CNNs) marked a paradigm shift by enabling end-to-end learning and automatic hierarchical feature extraction [2]. Unlike traditional ML pipelines, CNN-based models eliminate the need for manual feature design and are better suited to capturing complex visual patterns associated with plant diseases. Foundational architectures such as VGGNet and ResNet rapidly became standard benchmarks for image-based plant disease classification.

Several studies demonstrated that CNNs significantly outperform traditional ML methods in rice disease detection. However, most early CNN-based works evaluated a single architecture or a small subset of models on a specific dataset, often reporting high validation accuracy without extensive analysis of generalization performance. Consequently, while these studies confirmed the superiority of CNNs, they provided limited insight into how different architectures compare under identical experimental conditions—a limitation that the present work explicitly addresses through a systematic evaluation of ten diverse CNN backbones.

2.3 Advanced CNN and Attention-Based Models

More recent research has focused on improving both accuracy and efficiency through deeper and more sophisticated CNN architectures. The EfficientNet family, which employs compound scaling to balance network depth, width, and resolution, has shown strong performance in rice leaf disease classification. Padhi et al. [13] reported a test accuracy of 96.91% using EfficientNet-B4, demonstrating the effectiveness of modern CNN design principles.

Attention mechanisms have also been integrated into CNN architectures to enhance feature discrimination. Al-Gaashani et al. [18] proposed a ResNet50-based attention model that achieved 98.71% accuracy by emphasizing disease-relevant regions of leaf images. Hybrid approaches combining CNN feature extractors with traditional classifiers such as SVM have also been explored to leverage complementary strengths [1, 8]. While these advanced models report impressive performance, they are often

evaluated on balanced datasets and do not systematically analyze the ”generalization gap” on unseen data—an issue this study prioritizes by analyzing the drop between validation and test performance.

2.4 GAN-Based Data Augmentation

Class imbalance is a persistent issue in agricultural image datasets, where certain diseases occur far less frequently than others. To mitigate this problem, recent studies have explored Generative Adversarial Networks (GANs) for synthetic data augmentation. Lamba et al. [19] demonstrated that GAN-generated samples can improve classification performance by enriching minority classes. Among GAN variants, Wasserstein GANs with Gradient Penalty (WGAN-GP) have gained attention for their training stability and ability to generate high-fidelity synthetic images.

Despite their promise, GAN-based augmentation techniques are typically evaluated in isolation or in combination with a single classifier, limiting understanding of how different CNN architectures respond to synthetic data. Addressing this limitation is a key motivation for this research, where WGAN-GP-based augmentation has been incorporated as a distinct experimental pipeline to test its efficacy across a wide spectrum of ten different deep learning models.

2.5 Research Gap and Contribution of This Work

From the reviewed literature, several critical research gaps are evident. First, existing studies largely focus on individual models or limited architectural comparisons, making it difficult to draw robust conclusions about the relative strengths of different CNN architectures for rice leaf disease classification. Second, although data augmentation is widely adopted, its comparative impact—particularly between geometric augmentation and GAN-based class balancing—has not been systematically evaluated across multiple models. Third, many works emphasize high validation accuracy while overlooking the generalization gap between validation and unseen test data, which is essential for real-world applicability.

This study addresses these gaps by conducting a comprehensive and controlled comparison of ten prominent pre-trained CNN architectures across three distinct data preparation pipelines: an original imbalanced dataset, a geometrically augmented dataset, and a WGAN-GP–balanced dataset. By explicitly analyzing test-set performance and generalization gaps, this work demonstrates why architectural sophistication alone is insufficient without appropriate data balancing. The findings establish that the combination of a deep residual architecture (ResNet101) and GAN-based class balancing yields superior generalization, thereby providing a more reliable framework for practical rice leaf disease detection systems.

3 Methodology

The methodology of this study is designed as a systematic, multi-pipeline framework to rigorously evaluate the performance of various CNN architectures for rice leaf disease classification. The overall workflow, depicted in Figure 1, begins with data collection and preprocessing, followed by model training using a transfer learning approach, and concludes with a comprehensive evaluation of the models on unseen test data. Each component of this workflow is detailed in the subsequent sections.

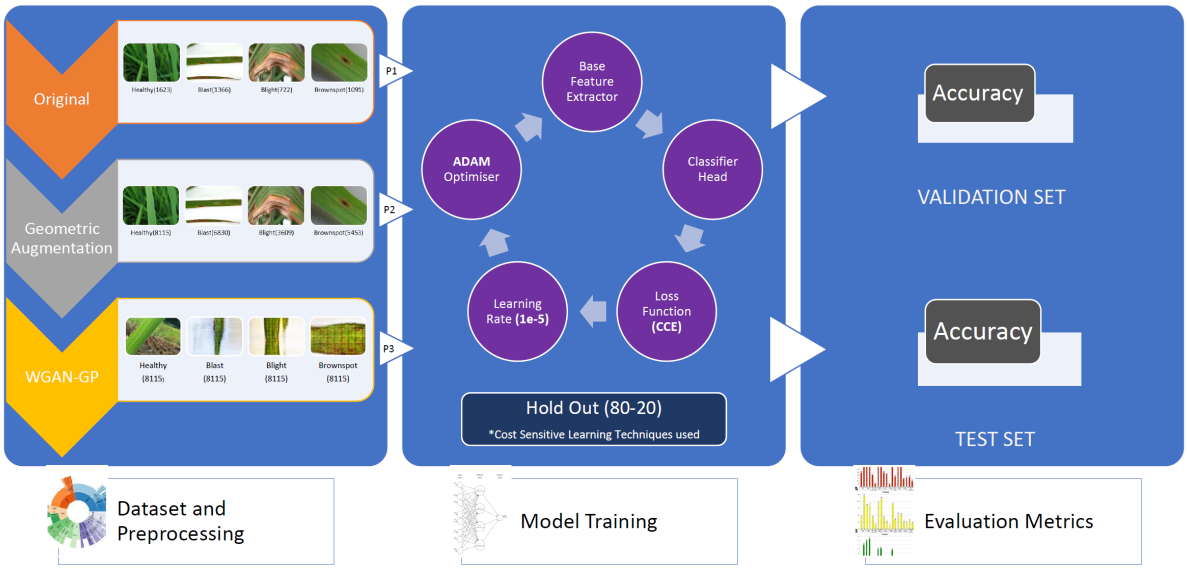


Figure 1: The overall workflow diagram of the proposed methodology, illustrating the three data preparation pipelines (P1, P2, P3), the model training process, and the final evaluation.

3.1 Dataset and Preprocessing

The foundation of this study is a dataset comprising images of rice leaves, categorized into four classes: Healthy, Bacterial Blight, Blast, and Brownspot, as shown in Figure 2. The data was aggregated from multiple publicly available repositories to ensure a diverse representation of disease symptoms across different environmental conditions and capture devices.

The primary source was the Mendeley Data repository [31], specifically the ”Rice Leaf Bacterial and Fungal Disease Dataset,” which provided high-resolution field images captured in Bangladesh using mobile devices. This was supplemented by the ”Rice Leaf Diseases Dataset” from Kaggle [32], which contributed images of bacterial leaf blight and brown spot collected from rice fields in Gujarat, India. Additional samples were incorporated from the ”Rice Leafs Disease Dataset” [33] and the ”Rice Leaf Diseases Detection” dataset [34], both of which offered curated collections of disease pathologies including Leaf Blast. Further diversity in lighting and background conditions was introduced through the ”Rice Disease Dataset” [35] and the ”RiceLeafs” dataset [36], which provided pre-partitioned images suitable for training robust classification models.

To investigate the impact of data quality and quantity on model performance, the experiments in this project were structured around three distinct data preparation pipelines.

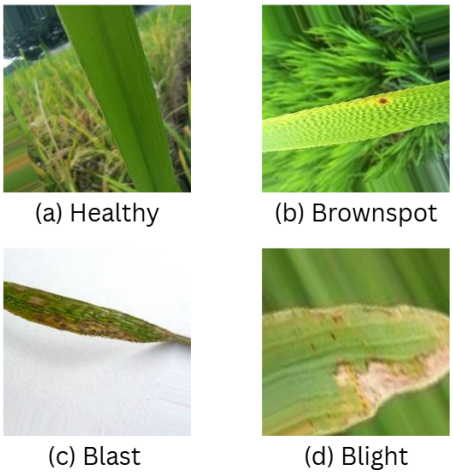


Figure 2: Sample images from the dataset representing the four classes:
(a) Healthy, (b) Brownspot, (c) Blast, and (d) Blight.

3.1.1 Pipeline 1: Original Imbalanced Dataset

The initial dataset consists of 4,802 raw images, exhibiting a significant class imbalance. The distribution, shown in Figure 3, is as follows: Healthy (1,623 images), Blast (1,366 images), Brownspot (1,091 images), and Blight (722 images). This imbalance, with a class ratio of approximately 2.25:1.89:1.51:1, reflects real-world scenarios where disease prevalence is often uneven. All images in this pipeline undergo standard preprocessing, including resizing to a uniform dimension of 224×224 pixels and normalization of pixel values. This specific resolution was selected because it is the native input size for the majority of ImageNet pre-trained architectures (such as VGG16 and ResNet). Adhering to this standard ensures that the spatial hierarchies learned during pre-training are preserved effectively, minimizing the artifacting or feature loss that can occur with arbitrary interpolation.

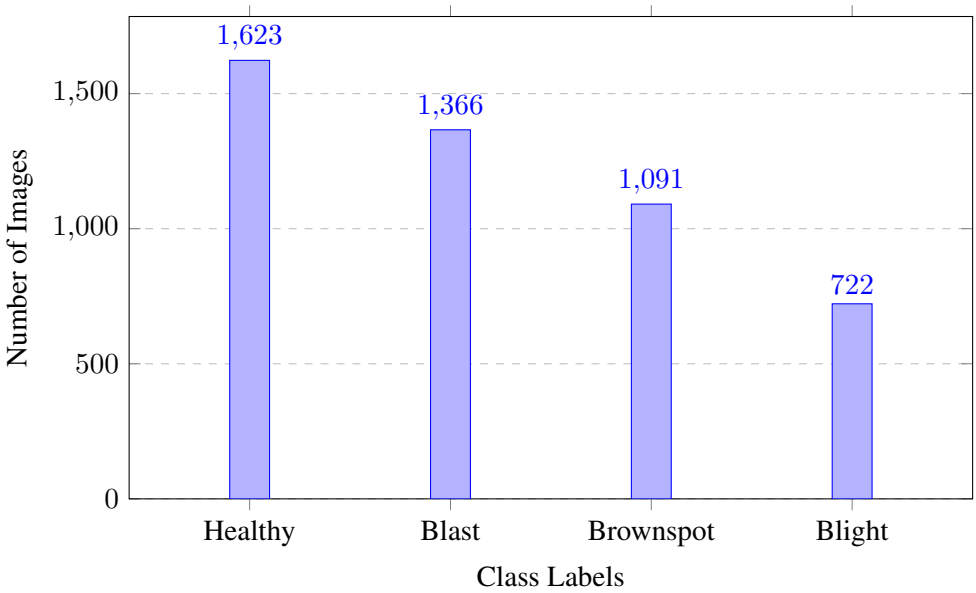


Figure 3: Class distribution of the original imbalanced dataset (Pipeline 1).

3.1.2 Pipeline 2: Geometrically Augmented Dataset

To address the issue of data scarcity, the second pipeline expands the original dataset through geometric augmentation. Basic transformations such as random flips, rotations, and shifts were applied to generate

five new images from each original image. This process increased the total dataset size to 23,907 images. As illustrated in Figure 4, this technique substantially increases the volume of training data but also amplifies the pre-existing class imbalance.

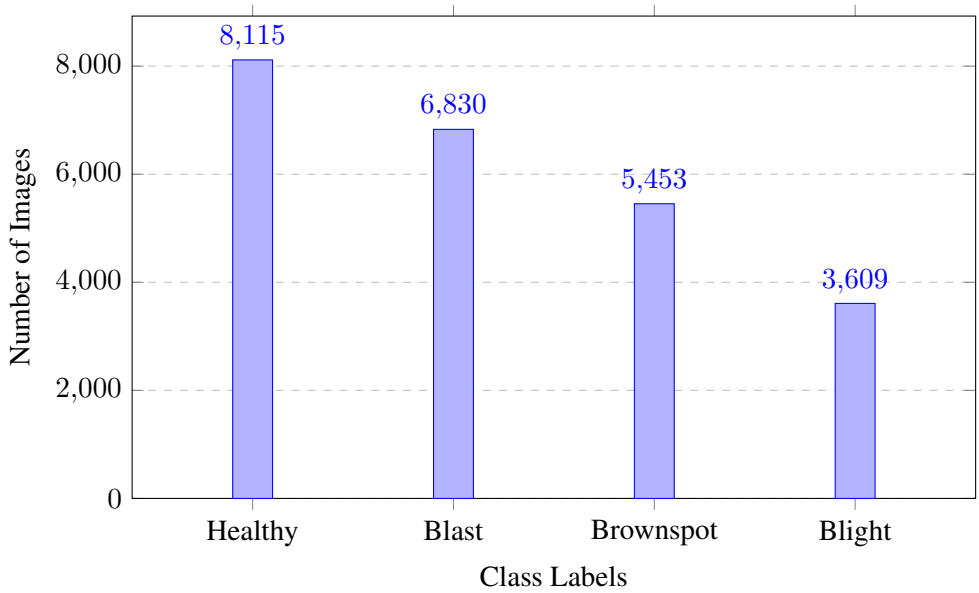


Figure 4: Class distribution of the geometrically augmented dataset (Pipeline 2).

3.1.3 Pipeline 3: Balanced Dataset with WGAN-GP

The third pipeline aims to create a large, balanced dataset by addressing the class imbalance amplified in Pipeline 2. A Wasserstein Generative Adversarial Network with Gradient Penalty (WGAN-GP) was employed to generate synthetic, high-realism images for the minority classes. As shown in Figure 5, WGAN-GP is a generative model where a generator network learns to create new data, and a discriminator network learns to distinguish real data from synthetic data. This adversarial process results in the generation of new images that are statistically similar to the original data. Synthetic images were generated for the Blast, Blight, and Brownspot classes to match the image count of the majority class (Healthy). This resulted in a perfectly balanced dataset with 8,115 images per class, totaling 32,460 images (Figure 6).

WGAN-GP Training Configuration The WGAN-GP models were trained individually for each minority class (Blast, Blight, and Brownspot) for 30 epochs using a batch size of 32. Adam Optimizer was used with a learning rate of 2×10^{-4}, $\beta_1 = 0.5$, and $\beta_2 = 0.9$ for both the generator and the critic. To ensure training stability and satisfy the Lipschitz constraint required for the Wasserstein distance, the gradient penalty method proposed by Gulrajani et al. [30] was implemented with a coefficient of $\lambda = 10$. The training process followed a strict update ratio where the critic network was updated $n_{critic} = 5$ times for every single generator update, allowing the critic to maintain an optimal gradient estimate.

Validation of Synthetic Image Quality The quality of the generated synthetic images was validated using a two-tiered approach. *Qualitative validation* was performed during training by inspecting generated image grids every 5 epochs to monitor for mode collapse and visual fidelity to the specific disease morphology (e.g., ensuring distinct spindle-shaped lesions for Blast). *Quantitative validation* was conducted extrinsically by assessing the downstream performance of the CNN classifiers; the significant increase in test accuracy observed in Pipeline 3 (as detailed in Section 4) confirms that the synthetic data successfully captured the underlying data distribution of the minority classes.

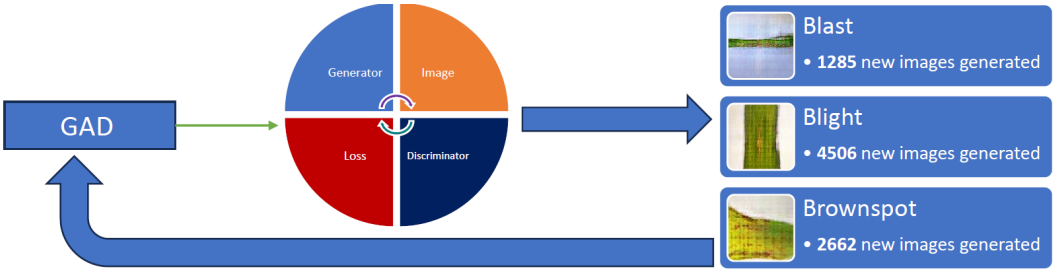


Figure 5: Illustration of the WGAN-GP process for generating synthetic images to balance the dataset. (GAD refers to Geometrically Augmented Dataset)

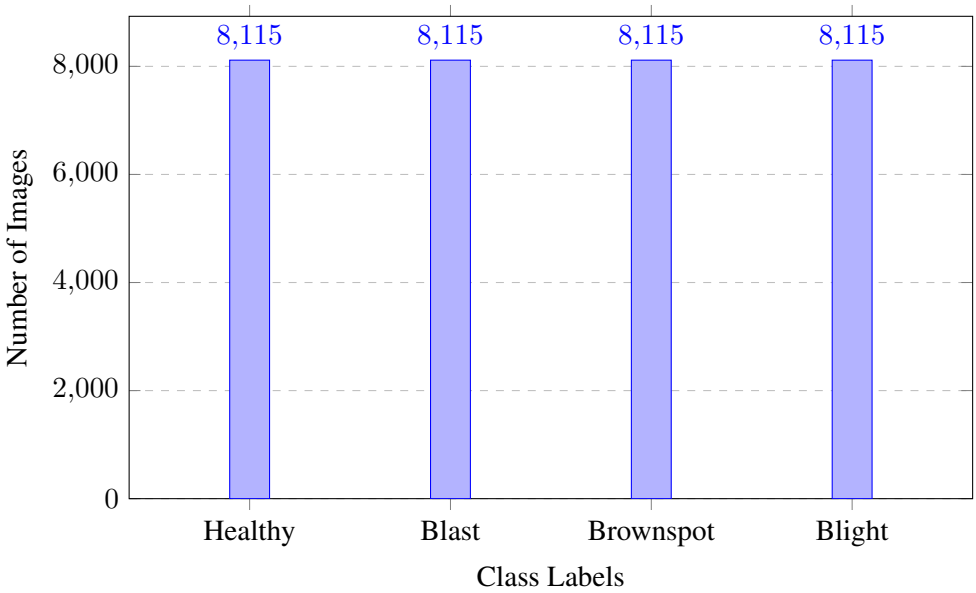


Figure 6: Class distribution of the balanced dataset after WGAN-GP augmentation (Pipeline 3).

3.2 Model Training

Our study employs a transfer learning approach to leverage the powerful feature extraction capabilities of pre-trained CNNs. The training process is standardized across all models and pipelines to ensure a fair comparison.

3.2.1 CNN Architectures as Feature Extractors

Ten widely recognized pre-trained CNN architectures were selected to serve as the base feature extractors. This selection was not arbitrary; the models were chosen to represent a diverse spectrum of deep learning design philosophies, ranging from standard deep networks to sophisticated residual and mobile-optimized architectures.

These include:

- **Standard Deep Networks:** VGG16 [20], representing early depth-focused architectures.
- **Residual Learners:** ResNet50 and ResNet101 [21], chosen to test the impact of skip connections and varying network depth.
- **Feature Reuse Architectures:** DenseNet121 [22], selected for its efficient feature propagation.

- **Lightweight/Mobile Models:** MobileNetV3 [25], NasNetMobile [27], and EfficientNetV2B0 [26], included to evaluate performance on resource-constrained devices suitable for field deployment.
- **Scalable Architecture Search:** NasNetLarge [27], included to assess the performance upper-bound of an architecture discovered through neural architecture search, distinct from its mobile counterpart.
- **Specialized Architectures:** Xception [23] and InceptionV3 [24], chosen for their advanced factorized convolutions.

Table 1 summarizes the ten models of this study, along with their parameter count, layer count, input size, and their key architectural features.

Table 1: Architectural Details of the Pre-trained CNN Models Used

Model	Parameters (Millions)	Depth (Layers)	Input Size	Key Architectural Feature
VGG16 [20]	138.4	16	224×224	Stacked 3×3 convolutions, high depth
ResNet50 [21]	25.6	50	224×224	Residual (skip) connections
ResNet101 [21]	44.7	101	224×224	Deeper residual connections
DenseNet121 [22]	8.1	121	224×224	Feature reuse via dense connections
MobileNetV3 [25]	5.4	Variable	224×224	Hardware-aware NAS, Squeeze-and-Excitation
NasNetMobile [27]	5.3	Variable	224×224	Scalable cells from Neural Architecture Search
NasNetLarge [27]	88.9	Variable	331×331	Scaled-up architecture search
EfficientNetV2B0 [26]	7.2	Variable	224×224	Faster training, progressive learning
Xception [23]	22.9	126	299×299	Depthwise separable convolutions
InceptionV3 [24]	23.9	159	299×299	Factorized convolutions, auxiliary classifiers

For each model, the pre-trained convolutional base was retained to leverage features learned from the large-scale ImageNet dataset, while the original top classification layers were removed.

3.2.2 Custom Classifier Head

A custom classifier head was appended to each feature extractor. This head consists of a sequence of layers designed to adapt the learned features to the specific classification task: a Dense layer with 512 neurons (ReLU), a Dropout layer (0.5), a Dense layer with 256 neurons (ReLU), and a final Dense output layer with 4 neurons (Softmax).

This specific architecture tapering from 512 to 256 neurons was designed to create a gradual bottleneck. Direct mapping from the high-dimensional feature vectors of the base models (which often exceed 2048 dimensions) to just 4 classes can lead to significant information loss. The intermediate dense layers allow for a progressive reduction in dimensionality, enabling the model to learn more abstract, non-linear combinations of the extracted features before final classification. The Dropout layer was introduced specifically to prevent overfitting within these fully connected layers.

3.2.3 Two-Phase Fine-Tuning Strategy

A two-phase training strategy, illustrated in Figure 7, was implemented to effectively fine-tune the models. The specific epoch counts for each phase were determined empirically to balance convergence speed with the risk of overfitting.

- **Phase 1 (30 epochs - Head Stabilization):** The weights of the pre-trained feature extractor were frozen, and only the randomly initialized weights of the custom classifier head were trained. The limit of 30 epochs was selected because the classifier head, having far fewer parameters than the backbone, converges relatively quickly. This phase serves as a "warm-up," ensuring that the gradients backpropagated in the next phase are stable and do not destroy the pre-trained features.

- **Phase 2 (60 epochs - Fine-Tuning):** All layers of the model were unfrozen and fine-tuned at a reduced learning rate (1×10^{-5}). A higher limit of 60 epochs was set for this phase because fine-tuning involves making minute adjustments to the deep feature maps, a process that requires more iterations to converge without causing catastrophic forgetting.

It is important to note that these epoch counts served as maximum upper bounds. The actual training duration for each model was dynamically controlled by the **Early Stopping** mechanism (patience = 4), which halted training automatically once the validation loss plateaued, thereby preventing unnecessary computation and overfitting.

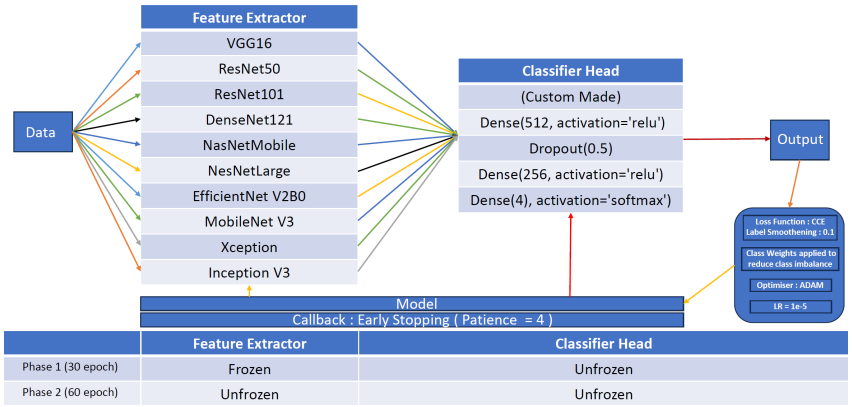


Figure 7: The two-phase fine-tuning strategy. In Phase 1, only the custom classifier head is trained. In Phase 2, the entire network is fine-tuned.

3.2.4 Training Parameters

To ensure consistency, all models were trained using the same set of hyperparameters. The ADAM optimizer was used with a learning rate of 1×10^{-5}. The loss function was Categorical Cross-Entropy (CCE), supplemented with label smoothing (0.1) to regularize the model. To mitigate the effects of class imbalance in Pipelines 1 and 2, class weights were applied during training, giving higher importance to minority classes. An Early Stopping callback with a patience of 4 epochs was used to prevent overfitting by halting training if the validation loss did not improve.

3.3 Model Evaluation Protocol

To ensure a rigorous assessment of model generalizability, a multi-stage evaluation protocol was adopted, which involved both an internal validation set and an independent test set.

3.3.1 Data Partitioning

The aggregated dataset (described in Section 3.1) was partitioned using an **80:20 ratio** for training and validation, respectively.

- **Training Set (80%):** Used to learn the model weights.
- **Validation Set (20%):** Used exclusively to monitor training progress, tune hyperparameters, and trigger the Early Stopping mechanism.

In addition to this split, a **separate, independent Test Set** was employed for the final performance benchmarking. This test set was strictly isolated from the training and validation processes to prevent data leakage and to provide an unbiased evaluation of the models' performance on truly unseen data.

3.3.2 Performance Metrics and Analysis

While accuracy served as a primary baseline, relying solely on it can be misleading, particularly in agricultural diagnosis where the cost of false negatives is high. Therefore, all models were evaluated using a comprehensive suite of metrics on both the Validation Set (VD) and the Test Set (TD):

To interpret these metrics, the fundamental aggregates of the confusion matrix are defined as:

- **TP (True Positives):** The number of images correctly identified as belonging to the specific disease class.
- **TN (True Negatives):** The number of images correctly identified as *not* belonging to the specific class.
- **FP (False Positives):** The number of images incorrectly classified into the specific disease class (Type I error).
- **FN (False Negatives):** The number of images belonging to the specific disease class but incorrectly classified as another category (Type II error).

The performance metrics derived from these aggregates are defined as follows:

$$Precision = \frac{TP}{TP + FP} \tag{1}$$

$$Recall = \frac{TP}{TP + FN} \tag{2}$$

$$F1\text{-}Score = 2 \times \frac{Precision \times Recall}{Precision + Recall} \tag{3}$$

Precision measures the proportion of positive identifications that were actually correct, while Recall (Sensitivity) measures the proportion of actual positives identified correctly. The F1-Score represents the harmonic mean of the two, providing a balanced metric crucial for assessing performance on imbalanced classes.

3.4 Confusion Matrix Analysis

Beyond scalar metrics, we visualized confusion matrices to qualitatively analyze the types of errors being made. This allowed us to identify specific inter-class conflicts (e.g., distinguishing between visually similar Brownspot and Blast lesions) that aggregate accuracy scores might obscure. By comparing these metrics across the Validation and Test sets, we were able to explicitly quantify the generalization gap reported in Section 4.

4 Results and Discussion

4.1 Experimental Setup

To ensure reproducibility and provide context for the performance metrics, all computational experiments were conducted in a standardized environment. The models were implemented using the TensorFlow 2.16.1 framework with Keras 3.10.0 and TF-Keras 2.16.0 backends. The software environment was managed via a Conda-based distribution running Python 3.11.13 on an Ubuntu 22.04 LTS operating system.

The hardware configuration utilized for model training, Generative Adversarial Network (GAN) synthesis, and comparative analysis included an Intel Core Ultra 9 185H processor and 32GB of DDR5 RAM. To accelerate deep learning computations and gradient updates, an NVIDIA GeForce RTX 4070 GPU with 8GB of VRAM was employed. Performance tracking and visualization were facilitated by

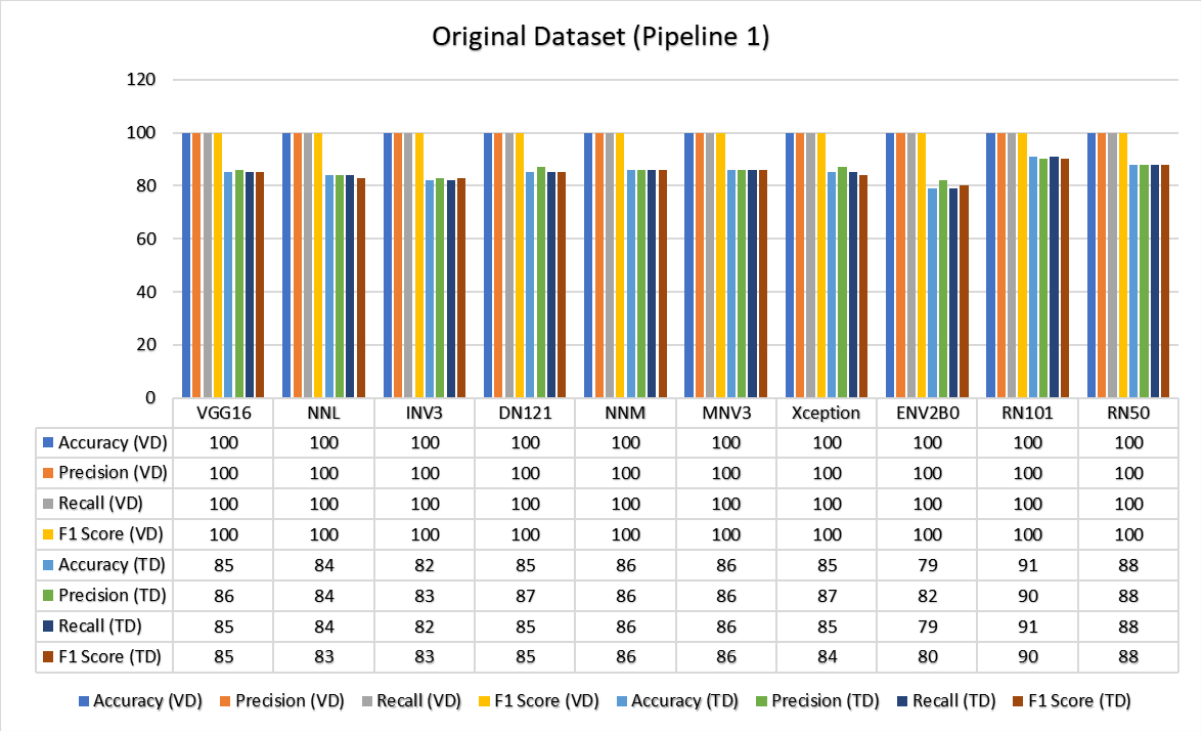
TensorBoard 2.16.2 and Matplotlib 3.8.4, while statistical evaluations and data preprocessing tasks were performed using Scikit-learn 1.4.2. All input data across the three pipelines were standardized to a batch size of 32, ensuring a consistent computational load and fair architectural comparison during the two-phase fine-tuning process.

4.2 Experimental Performance Analysis

The ten CNN architectures were rigorously evaluated across the three experimental pipelines. A consistent observation across all experiments was a significant generalization gap: while models frequently achieved 99–100% accuracy on the validation set, their performance on the unseen test set was considerably lower, ranging from 79% to 92%. This highlights the challenge of model generalization in this domain.

4.2.1 Performance on the Original Dataset (Pipeline 1)

Training on the original, imbalanced dataset (Pipeline 1) provided a baseline for model performance. In this setting, **ResNet101** distinguished itself as the top-performing architecture, achieving a test accuracy of **91%** and an F1-score of 90%. This indicates its strong inherent capability for feature extraction even under sub-optimal data conditions. Other models, including ResNet50 (88%) and NNM (86%), also demonstrated robust performance. Conversely, the 79% accuracy of ENV2B0 suggests that certain architectures are more susceptible to the learning bias introduced by class imbalance. Figure 8 shows the results yielded by each CNN Model for Pipeline 1.

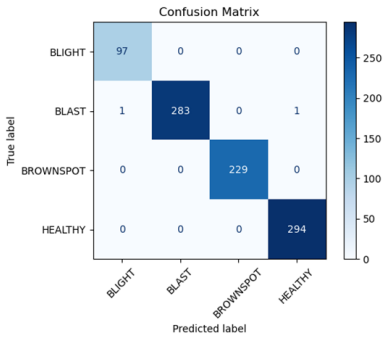


	VGG16	NNL	INV3	DN121	NNM	MNV3	Xception	ENV2B0	RN101	RN50
Accuracy (VD)	100	100	100	100	100	100	100	100	100	100
Precision (VD)	100	100	100	100	100	100	100	100	100	100
Recall (VD)	100	100	100	100	100	100	100	100	100	100
F1 Score (VD)	100	100	100	100	100	100	100	100	100	100
Accuracy (TD)	85	84	82	85	86	86	85	79	91	88
Precision (TD)	86	84	83	87	86	86	87	82	90	88
Recall (TD)	85	84	82	85	86	86	85	79	91	88
F1 Score (TD)	85	83	83	85	86	86	84	80	90	88

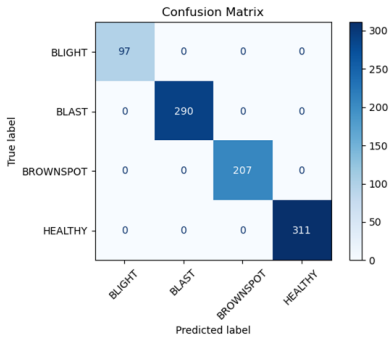
Figure 8: Comprehensive Test Performance Metrics of All CNN Models for Pipeline 1. (Here VD refers to Validation Dataset and TD refers to Test Dataset.)

The performance of each CNN Model has been recorded in form of confusion matrices, both for the *Validation Set* and *Test Set* in Figures 9 and 10 respectively. The saturation of the color of the cell is directly related to the number of images of each true–predicted class pair: darker (more saturated) cells indicate a higher number of samples, while lighter cells indicate fewer samples.

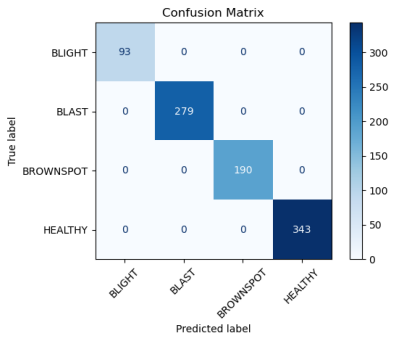
Consequently, the diagonal cells—where the predicted label matches the true label—represent correct classifications and ideally appear the most saturated. Off-diagonal cells represent misclassifications, and their intensity highlights which classes the model most frequently confuses.



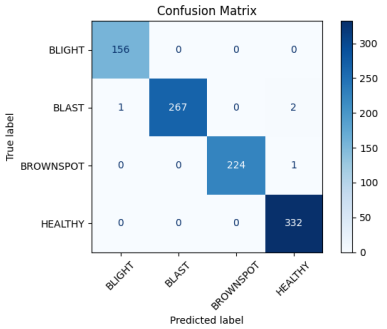
1. VGG16



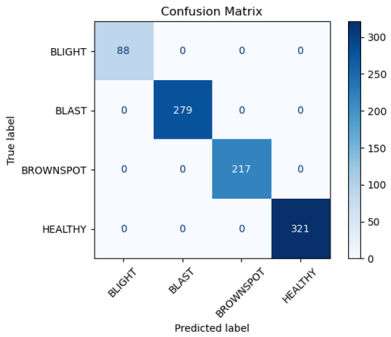
2. ResNet50



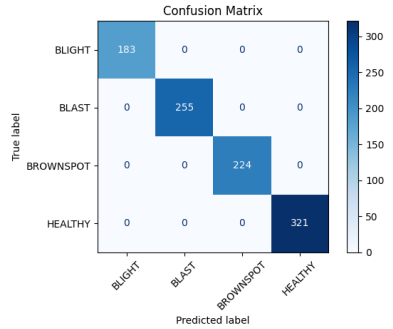
3. ResNet101



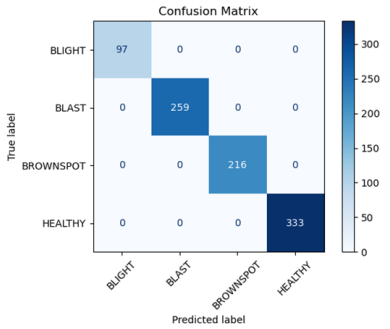
4. DenseNet121



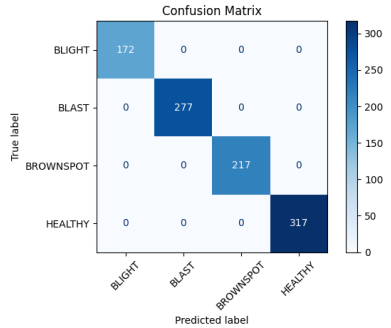
5. InceptionV3



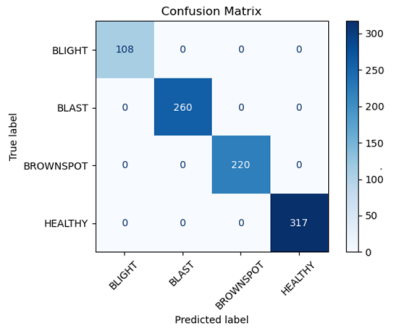
6. Xception



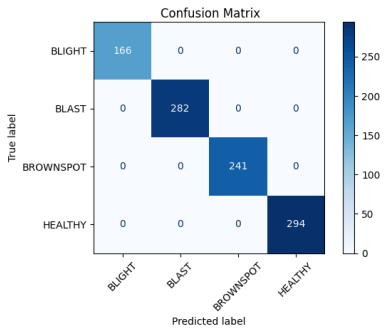
7. NasNetLarge



8. NasNetMobile

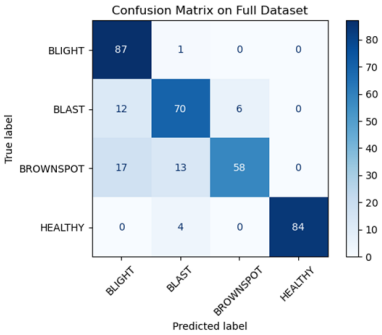


9. EfficientNetV2B0

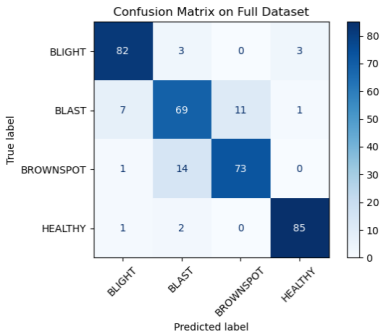


10. MobileNetV3 Validation

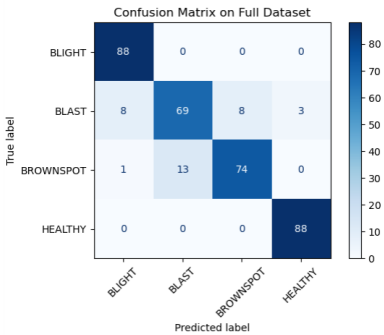
Figure 9: Confusion Matrices for Validation Set Performance of each CNN architecture evaluated on the Original Imbalanced Dataset (Pipeline 1)



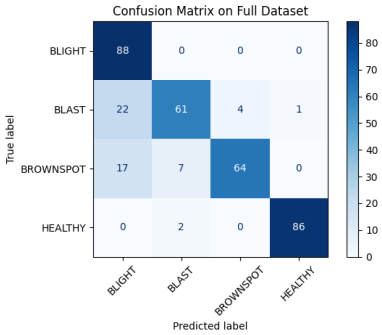
1. VGG16



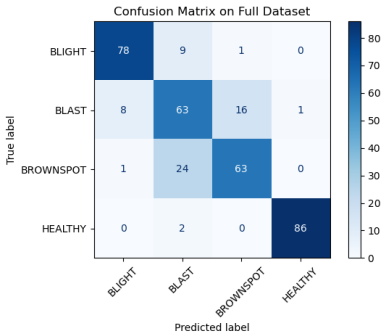
2. ResNet50



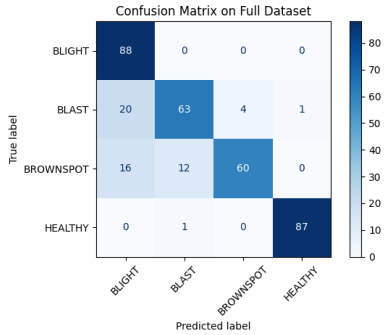
3. ResNet101



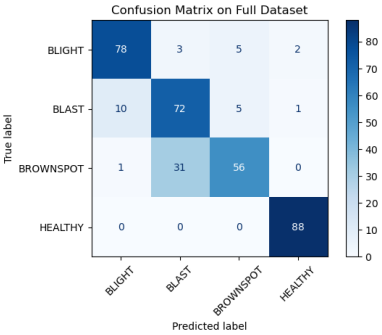
4. DenseNet121



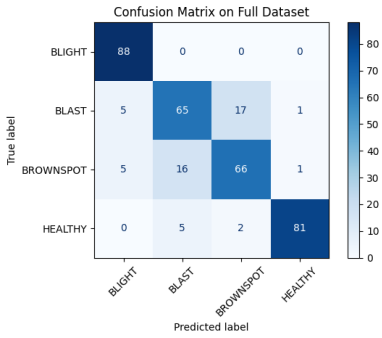
5. InceptionV3



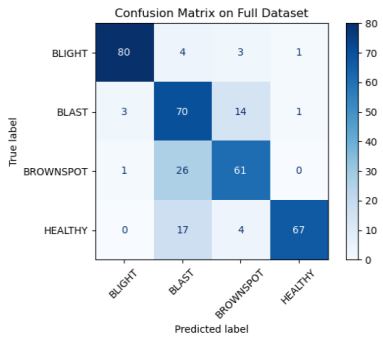
6. Xception



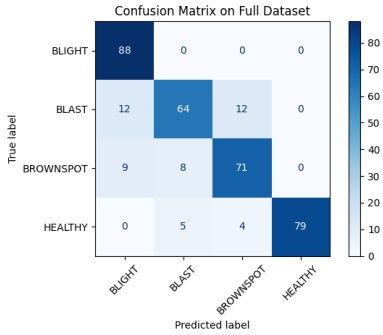
7. NasNetLarge



8. NasNetMobile



9. EfficientNetV2B0

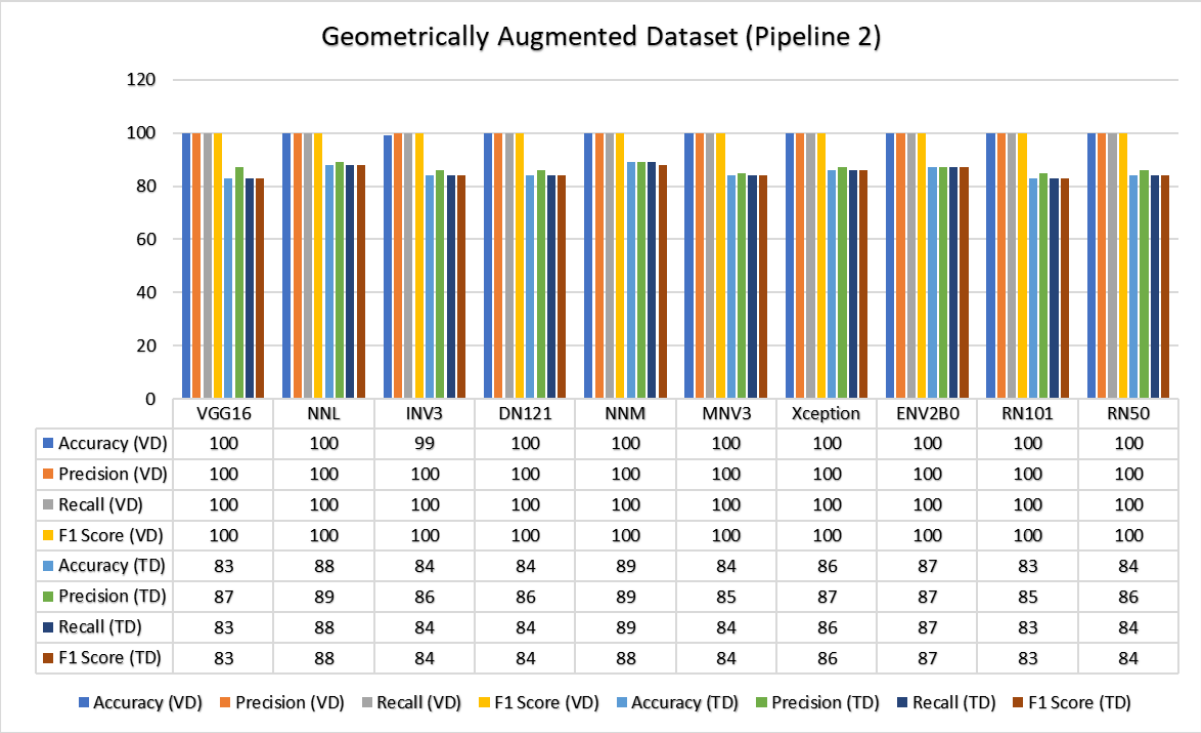


10. MobileNetV3 Validation

Figure 10: Confusion Matrices for Test Set Performance of each CNN architecture evaluated on the Original Imbalanced Dataset (Pipeline 1)

4.2.2 Impact of Geometric Augmentation (Pipeline 2)

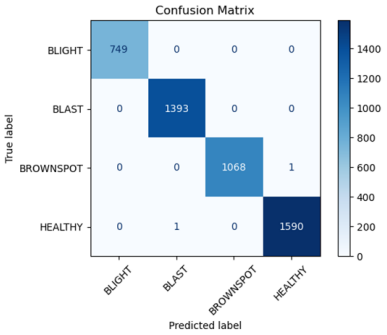
The introduction of geometric augmentations (Pipeline 2) yielded inconsistent results across architectures. While it successfully boosted the performance of models like **NNM** (from 86% to **89%**) and **ENV2B0** (from 79% to 87%), it proved detrimental to others. Most notably, the top-performing **ResNet101** experienced a significant drop in accuracy from 91% to **83%**. This counter-intuitive result suggests that naively increasing the volume of data without addressing the fundamental class imbalance can amplify the existing bias, leading models to overfit to the features of the now even more dominant majority classes. Figure 11 shows the results yielded by each CNN Model for Pipeline 2.



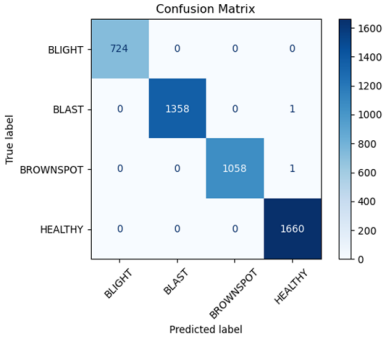
	VGG16	NNL	INV3	DN121	NNM	MNV3	Xception	ENV2B0	RN101	RN50
Accuracy (VD)	100	100	99	100	100	100	100	100	100	100
Precision (VD)	100	100	100	100	100	100	100	100	100	100
Recall (VD)	100	100	100	100	100	100	100	100	100	100
F1 Score (VD)	100	100	100	100	100	100	100	100	100	100
Accuracy (TD)	83	88	84	84	89	84	86	87	83	84
Precision (TD)	87	89	86	86	89	85	87	87	85	86
Recall (TD)	83	88	84	84	89	84	86	87	83	84
F1 Score (TD)	83	88	84	84	88	84	86	87	83	84

Figure 11: Comprehensive Test Performance Metrics of All CNN Models for Pipeline 2. (Here VD refers to Validation Dataset and TD refers to Test Dataset.)

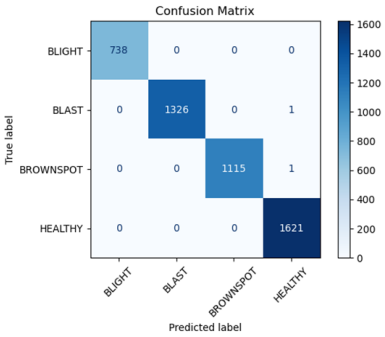
The performance of each CNN Model has been recorded in form of confusion matrices, both for the *Validation Set* and *Test Set* in Figures 12 and 13 respectively. Same as before, off-diagonal cells represent misclassifications, and their intensity highlights which classes the model most frequently confuses.



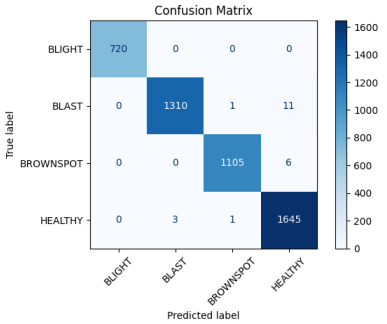
1. VGG16



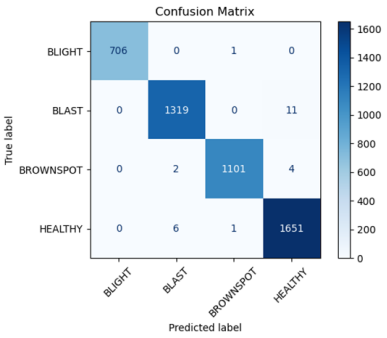
2. ResNet50



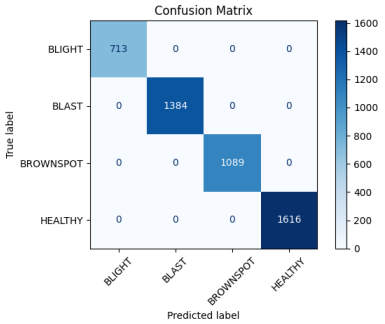
3. ResNet101



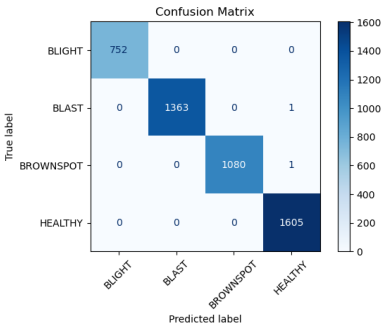
4. DenseNet121



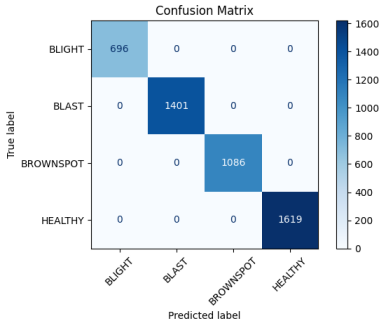
5. InceptionV3



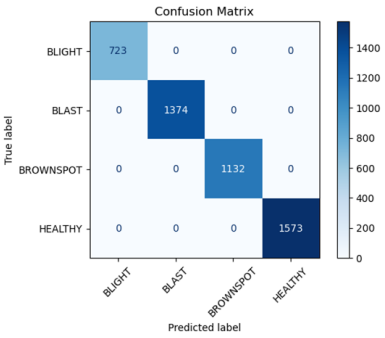
6. Xception



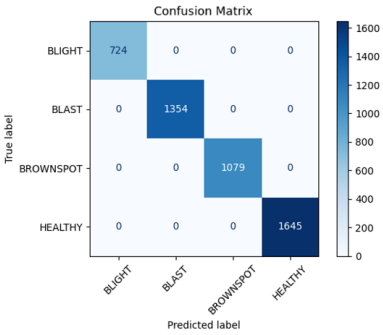
7. NasNetLarge



8. NasNetMobile

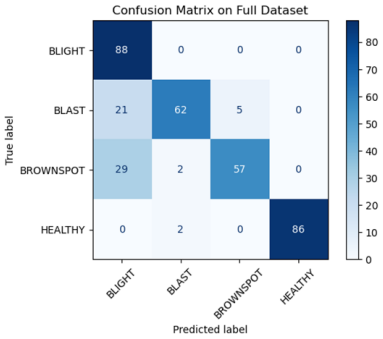


9. EfficientNetV2B0

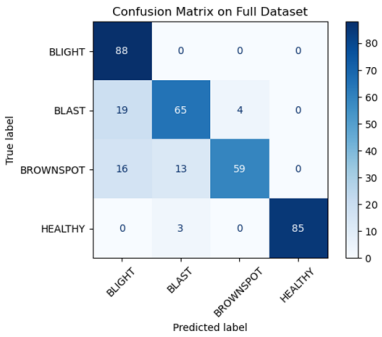


10. MobileNetV3 Validation

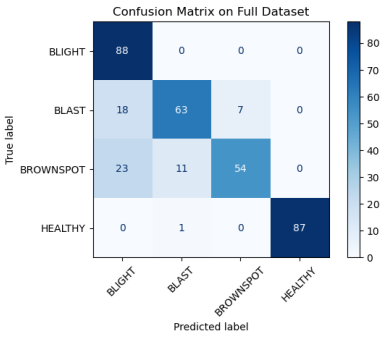
Figure 12: Confusion Matrices for Validation Set Performance of each CNN architecture evaluated on the Geometrically Augmented Dataset (Pipeline 2)



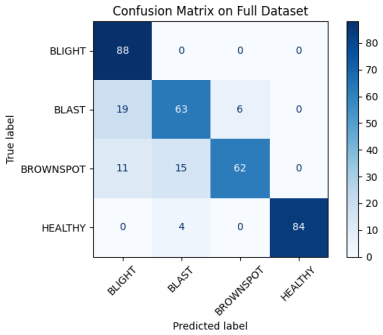
1. VGG16



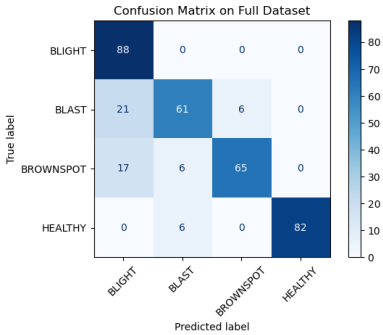
2. ResNet50



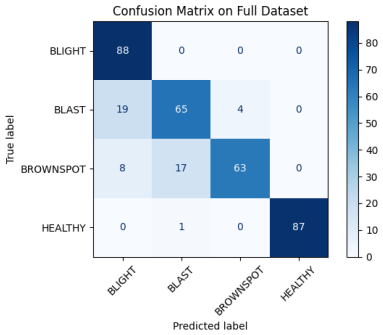
3. ResNet101



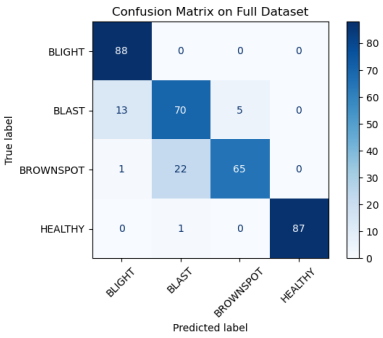
4. DenseNet121



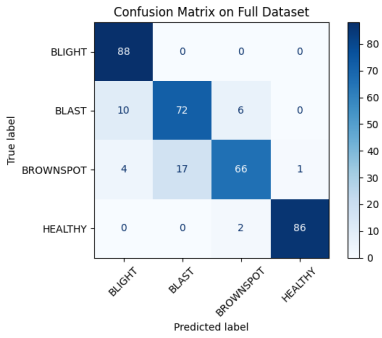
5. InceptionV3



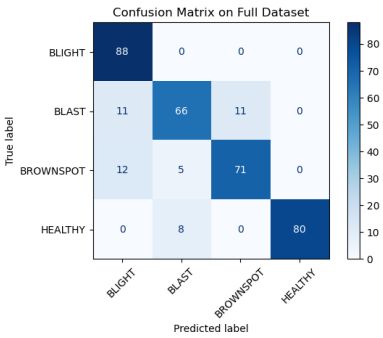
6. Xception



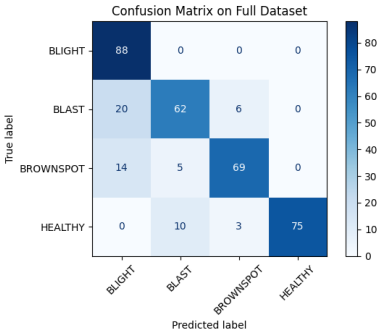
7. NasNetLarge



8. NasNetMobile



9. EfficientNetV2B0

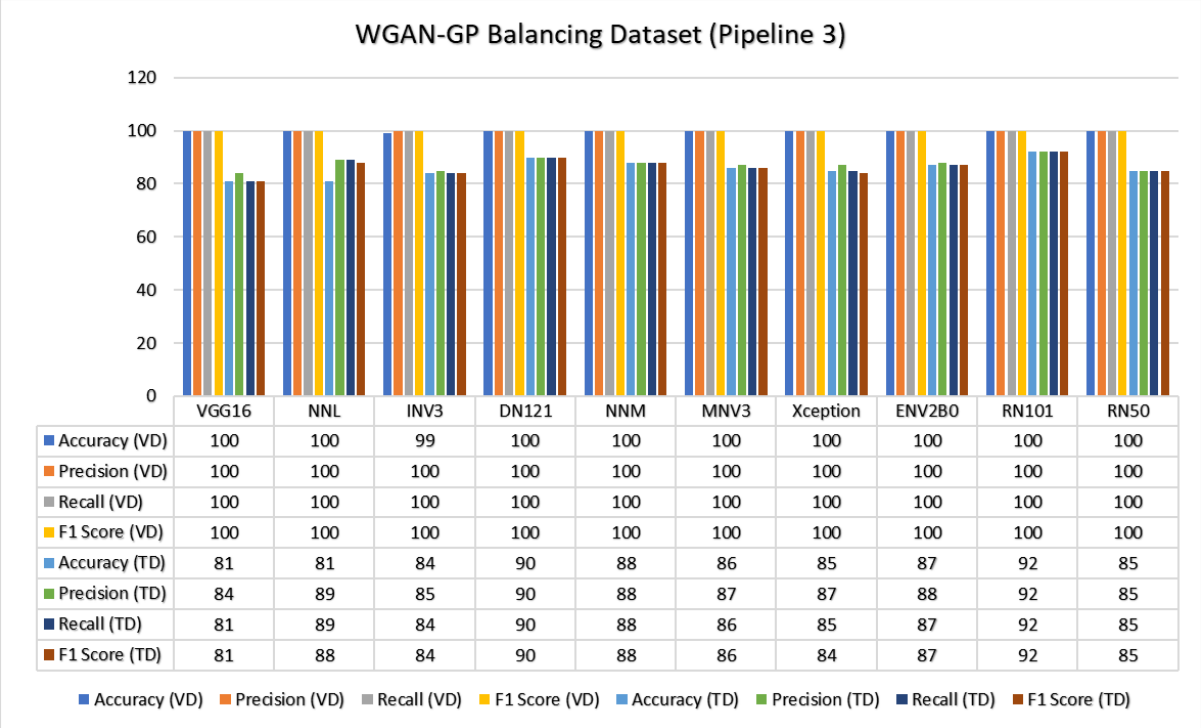


10. MobileNetV3 Validation

Figure 13: Confusion Matrices for Test Set Performance of each CNN architecture evaluated on the Geometrically Augmented Dataset (Pipeline 2)

4.2.3 Effect of WGAN-GP Balancing (Pipeline 3)

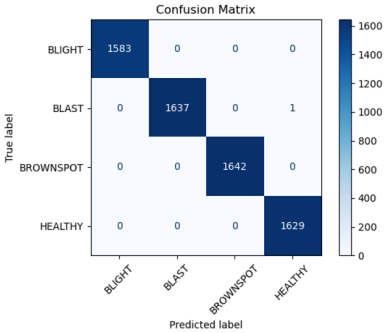
This pipeline, which introduced synthetic data to create a perfectly balanced training set (Pipeline 3), proved to be the most effective strategy for the top-tier models. By providing sufficient examples of minority classes, the models could learn more discriminative and generalizable features. **ResNet101** achieved its peak performance here, reaching a test accuracy, precision, recall, and F1-Score of **92%**. Similarly, **DenseNet121** showed a marked improvement, with its accuracy rising from 84% to **90%**. These results strongly validate the hypothesis that rectifying class imbalance is a critical precursor to achieving optimal model performance and generalization in this classification task. Figure 14 shows the results yielded by each CNN Model for Pipeline 3.



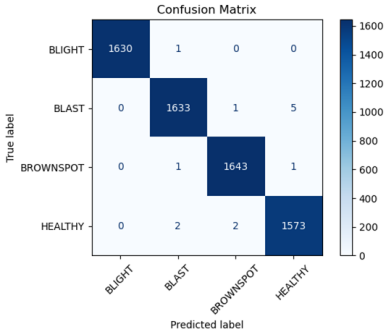
	VGG16	NNL	INV3	DN121	NNM	MNV3	Xception	ENV2B0	RN101	RN50
Accuracy (VD)	100	100	99	100	100	100	100	100	100	100
Precision (VD)	100	100	100	100	100	100	100	100	100	100
Recall (VD)	100	100	100	100	100	100	100	100	100	100
F1 Score (VD)	100	100	100	100	100	100	100	100	100	100
Accuracy (TD)	81	81	84	90	88	86	85	87	92	85
Precision (TD)	84	89	85	90	88	87	87	88	92	85
Recall (TD)	81	89	84	90	88	86	85	87	92	85
F1 Score (TD)	81	88	84	90	88	86	84	87	92	85

Figure 14: Comprehensive Test Performance Metrics of All CNN Models for Pipeline 3.
(Here VD refers to Validation Dataset and TD refers to Test Dataset.)

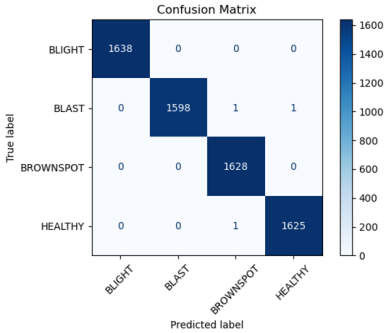
The performance of each CNN Model has been recorded in form of confusion matrices, both for the *Validation Set* and *Test Set* in Figures 15 and 16 respectively. It can be noted that there is now a reduction in false negatives across minority classes, as evident from the confusion matrices.



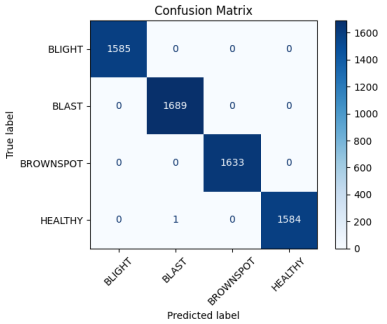
1. VGG16



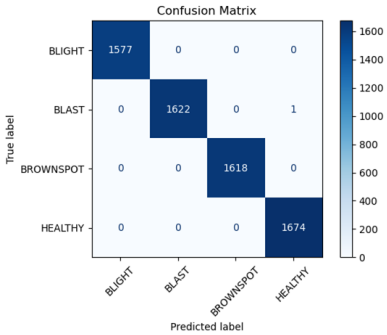
2. ResNet50



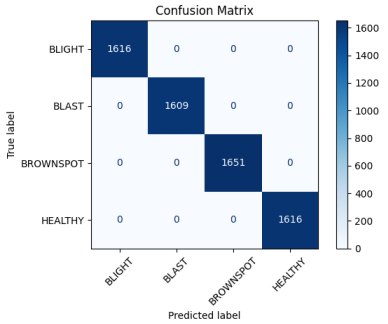
3. ResNet101



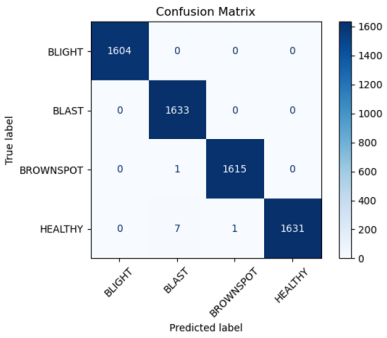
4. DenseNet121



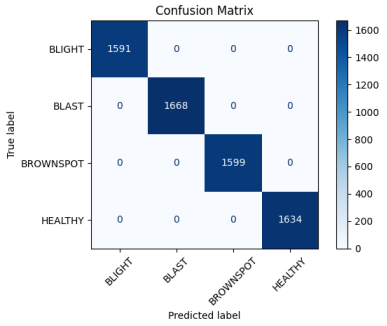
5. InceptionV3



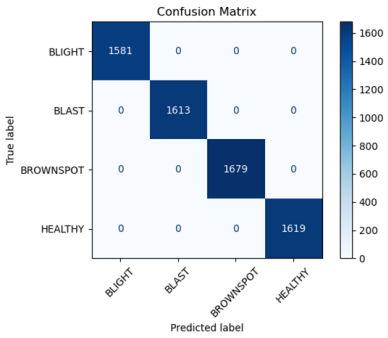
6. Xception



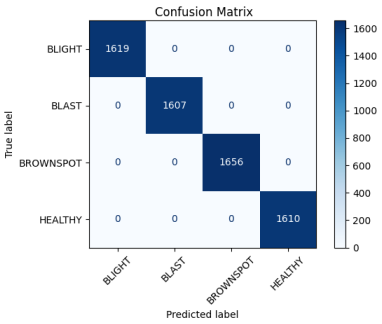
7. NasNetLarge



8. NasNetMobile

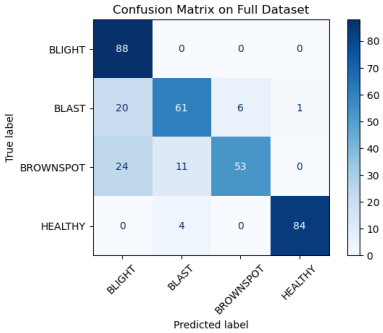


9. EfficientNetV2B0

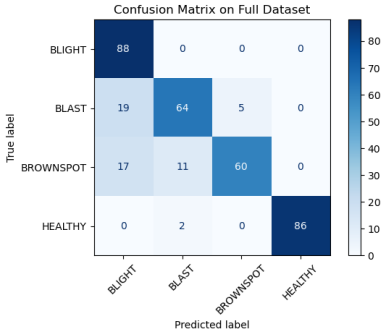


10. MobileNetV3 Validation

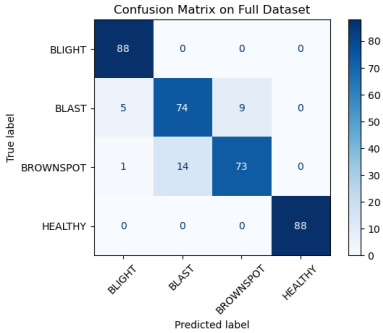
Figure 15: Confusion Matrices for Validation Set Performance of each CNN architecture evaluated on the WGAN-GP Balanced Dataset (Pipeline 3)



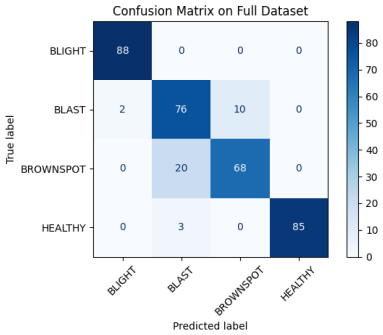
1. VGG16



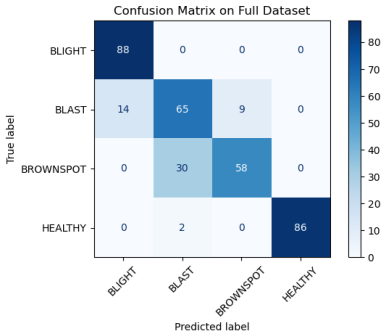
2. ResNet50



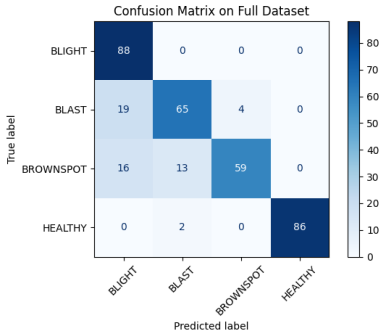
3. ResNet101



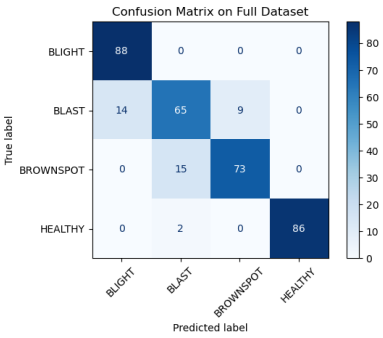
4. DenseNet121



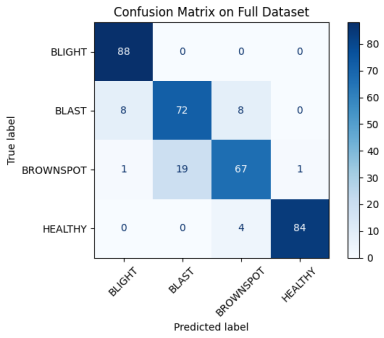
5. InceptionV3



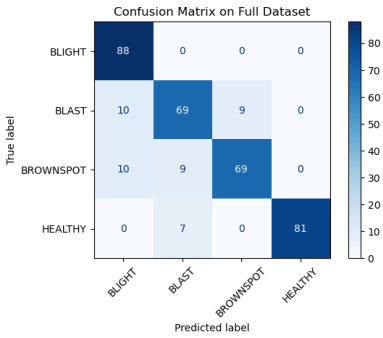
6. Xception



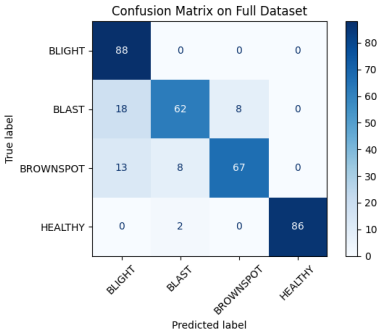
7. NasNetLarge



8. NasNetMobile



9. EfficientNetV2B0



10. MobileNetV3 Validation

Figure 16: Confusion Matrices for Test Set Performance of each CNN architecture evaluated on the WGAN-GP Balanced Dataset (Pipeline 3)

Finally, Figure 17 presents the comparison of the test accuracy (%) for top-performing models across the three augmentation pipelines.

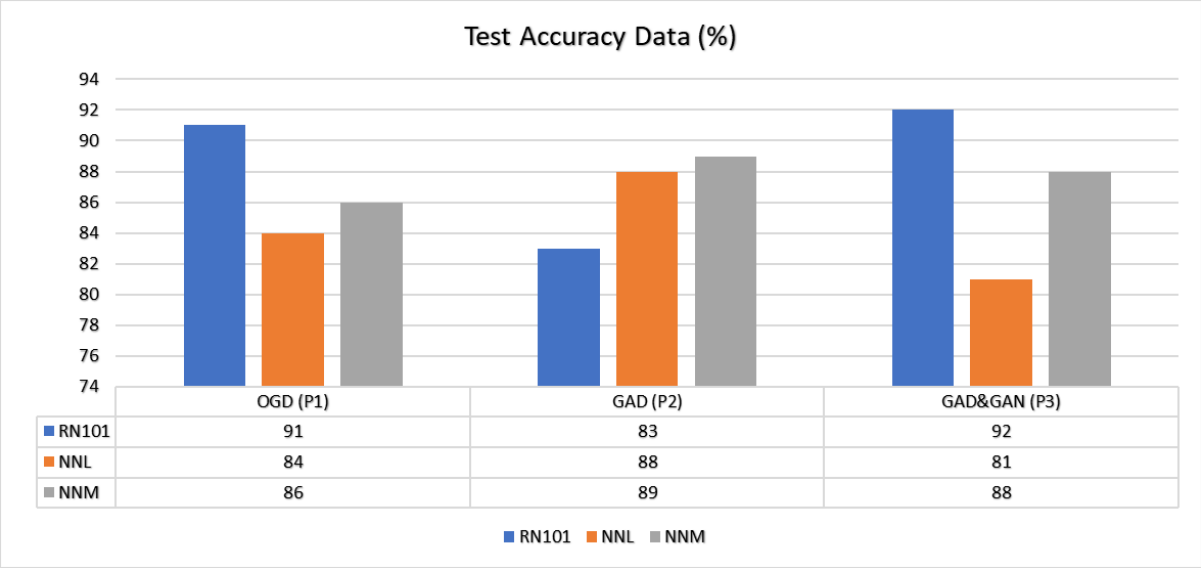


Figure 17: A Comparison Of The Top-Performing Models

4.3 Discussion of Findings

The experimental outcomes of this study provide several critical insights into the development of robust deep learning models for rice leaf disease classification. The results clearly indicate that an optimal solution arises from a synergistic combination of a suitable model architecture and a meticulously crafted data preparation strategy.

4.3.1 The Primacy of Class Balancing

The most unequivocal finding is the decisive impact of class balancing. The significant performance leap observed in Pipeline 3, where WGAN-GP was used to create a balanced dataset, underscores that addressing data imbalance is a more potent strategy than merely expanding the dataset with geometric augmentations. For the premier architectures, ResNet101 and DenseNet121, this pipeline represented the ”sweet spot,” enabling them to learn the distinguishing features of underrepresented classes (like Blight) without being biased by the majority classes. This confirms that for agricultural datasets, where class imbalance is a common challenge, advanced balancing techniques like synthetic data generation are not just beneficial but often essential for achieving high generalization performance.

4.3.2 Architectural Considerations: The Efficacy of ResNet101

While data strategy was paramount, architectural choice remained a significant factor. ResNet101’s consistent high performance, culminating in 92% accuracy, can be attributed to its deep structure and, critically, its use of residual connections. These connections facilitate gradient flow through the deep network, mitigating the vanishing gradient problem and allowing the model to learn more complex and effective feature hierarchies without performance degradation. In contrast, simpler architectures like VGG16, which lack such mechanisms, showed a steady decline as data complexity increased. The strong showing of DenseNet121 (90%), which encourages feature reuse across layers, further reinforces the conclusion that deep architectures with sophisticated mechanisms for managing information flow are best suited for fine-grained visual classification tasks like plant disease identification.

4.3.3 The Challenge of Generalization

A persistent observation throughout the study was the gap between near-perfect validation accuracy and the more modest test accuracy. This generalization gap indicates that while the models excelled at learn-

ing the patterns within the training distribution (including the validation split), their ability to generalize to entirely new, unseen data was constrained. This is a classic symptom of overfitting, albeit to the entire training dataset's characteristics. While the Geometrically augmented plus GAN generated pipeline (Pipeline 3) narrowed this gap for the top models, its persistence suggests that building a truly field-deployable model requires training data that captures an even greater diversity of real-world conditions, including different lighting, camera types, plant genotypes, and geographic locations.

5 Conclusion and Future Scope

5.1 Conclusion

This study conducted a comprehensive comparative analysis of ten prominent pre-trained Convolutional Neural Network (CNN) architectures for the classification of rice leaf diseases into four categories: Healthy, Bacterial Blight, Blast, and Brownspot. These models were rigorously evaluated under three distinct data pipelines: an original imbalanced dataset (Pipeline 1), a dataset expanded with geometric augmentation (Pipeline 2), and a dataset balanced using synthetic images from a WGAN-GP (Pipeline 3).

The findings reveal that the ResNet101 architecture, when trained on the dataset balanced with WGAN-GP, achieved superior performance, producing the highest test accuracy of 92%. This result strongly indicates that addressing class imbalance with advanced generative augmentation is more effective than relying on simple geometric transformations. A critical insight from this work is the identification of a significant generalization gap. Although most models, particularly on the augmented datasets, achieved near-perfect validation accuracies (99-100%), their performance on the unseen test set was considerably lower (79-92%). This discrepancy underscores that models can easily overfit to the validation set and highlights the critical importance of robust data augmentation strategies in developing systems that can generalize effectively to new, real-world data.

5.2 Future Scope

Based on the results and limitations of the present study, we propose several avenues for future research:

- **Bridging the Generalization Gap:** The observed gap, even in the best-performing model, suggests that future work should focus on collecting more diverse, in-field data. This would create a more challenging and realistic test bed to improve model robustness, moving towards practical application in smart-farming solutions.
- **Mobile Deployment and Optimization:** A key practical step is to deploy the top-performing model into a mobile application for farmers. This would require model optimization to create a lightweight, efficient model that can run on-device. This involves techniques such as:
  - **Quantization:** This process involves converting the model's 32-bit floating-point weights and activations to lower-precision formats, such as 8-bit integers (INT8). This can reduce the model size by up to 4x and significantly speed up inference on compatible mobile hardware.
  - **Pruning:** This technique systematically removes redundant or non-impactful connections (weights) within the network, creating a "sparse" model that requires fewer computations, further reducing latency and power consumption.
- **Exploring Advanced Architectures:** While this study covered many standard CNNs, future research could investigate the performance of more recent architectures, such as Vision Transformers (ViTs) [29], which have shown promising results in various computer vision tasks.
- **Enhancing Generative Augmentation:** The success of WGAN-GP encourages further exploration of advanced generative models. Future studies could compare WGAN-GP against other

techniques, such as diffusion models [28], to assess their impact on generating high-fidelity, diverse synthetic training images.

- **Expanding from Classification to Localization:** This work could be extended beyond simple classification to tasks like object detection or **semantic segmentation**. Using object detection models, such as YOLO [3], would provide more granular information. This would allow the system to not only identify the disease but also draw a bounding box around the affected area, quantifying the severity of the infection and offering farmers more actionable insights for targeted treatment.

References

[1] Sobiyaa, P., Jayareka, K. S., Maheshkumar, K., Somasundaram N., & Rao, K. S. (2022). Paddy disease classification using machine learning technique. *Materials Today: Proceedings, 64*, 883–887. DOI:10.1016/j.matpr.2022.05.398

[2] Tasfe, M., Nivrito, A., Al Machot, F., Ullah, M., & Ullah, H. (2024). Deep Learning Based Models for Paddy Disease Identification and Classification: A Systematic Survey. *IEEE Access, 12*, 100862–100891. DOI:10.1109/ACCESS.2024.3419708

[3] Yang, B., et al. (2023). Research on rice disease recognition based on improved YOLOv5. *PLoS ONE 18(12): e0295661*. DOI:10.1371/journal.pone.0295661

[4] Childs, N. (2010). Price Spikes in Global Rice Markets Benefit U.S. Growers. *Amber Waves*. USDA ERS. Link

[5] Chen, J., et al. (2020). Using deep transfer learning for image-based plant disease identification. *Computers and Electronics in Agriculture, 173*, 105393. DOI:10.1016/j.compag.2020.105393

[6] Khan, S. U., et al. (2025). A review on automated plant disease detection. *Journal of King Saud University – Computer and Information Sciences, 37(3)*, 101737. DOI:10.1007/s44443-025-00040-3

[7] Salka, T. D., et al. (2025). Plant leaf disease detection and classification using CNNs: A review. *Artificial Intelligence Review, 58(10)*. DOI:10.1007/s10462-025-11234-6

[8] Haridasan, A., et al. (2022). Deep learning system for paddy plant disease detection and classification. *Environ Monit Assess, 195(1)*, 120. DOI:10.1007/s10661-022-10656-x

[9] Alfarisy, A. A., et al. (2018). Deep learning-based classification for paddy pests and diseases. In *Proc. ICMAI*. DOI:10.1145/3208788.3208795

[10] Shrivastava, V., & Pradhan, M. K. (2020). Rice plant disease classification using color features. *Journal of Plant Pathology, 103*, 17–26. DOI:10.1007/s42161-020-00683-3

[11] Nancy, C., & Kiran, S. (2024). Cucumber Leaf Disease Detection using GLCM Features with Random Forest Algorithm. *Int. Res. J. of Multidisciplinary Technovation, 6(1)*, 40–50. DOI:10.54392/irjmt2414

[12] Rangarajan, A. K., et al. (2018). Tomato crop disease classification using pre-trained deep learning models. *Procedia Computer Science, 133*, 1040–1047. DOI:10.1016/j.procs.2018.07.070

[13] Padhi, J., et al. (2024). Paddy Leaf Disease Classification Using EfficientNet B4 with Compound Scaling and Swish Activation. *IEEE Access, 12*, 126426–126437. DOI:10.1109/ACCESS.2024.3451557

[14] Karthik, R., et al. (2020). Attention embedded residual CNN for disease detection in tomato leaves. *Applied Soft Computing, 86*, 105933. DOI:10.1016/j.asoc.2019.105933

[15] Sujatha, K., et al. (2023). UGVs for agri spray with AI-assisted paddy crop disease identification. *Procedia Computer Science, 230(3)*, 70–81. DOI:10.1016/j.procs.2023.12.062

[16] Ullah, E., et al. (2022). Deep learning-based wheat ears count in robot images for wheat phenotyping. *Electronic Imaging, 34(6)*, 264–1–264–6. DOI:10.2352/EI.2022.34.6.IRIACV-264

[17] Prajapati, H. B., Shah, J. P., & Dabhi, V. K. (2017). Detection and classification of rice plant diseases. *Intelligent Decision Technologies, 11*(3), 357-373. DOI:10.3233/IDT-170301

[18] Al-Gaashani, M. S. A. M., et al. (2023). Using a Resnet50 with a Kernel Attention Mechanism for Rice Disease Diagnosis. *Life, 13*(6), 1277. DOI:10.3390/life13061277

[19] Lamba, S., Baliyan, A., & Kukreja, V. (2022). Generative Adversarial Networks based Data Augmentation for Paddy Disease Detection using Support Vector Machine. In *2022 10th International Conference on Reliability, Infocom Technologies and Optimization (ICRITO)*. DOI:10.1109/ICRITO56286.2022.9964506

[20] Simonyan, K., & Zisserman, A. (2014). Very deep convolutional networks for large-scale image recognition. arXiv preprint arXiv:1409.1556

[21] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 770-778).DOI: 10.1109/CVPR.2016.90

[22] Huang, G., Liu, Z., Van Der Maaten, L., & Weinberger, K. Q. (2017). Densely connected convolutional networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 4700-4708).DOI: 10.1109/CVPR.2017.243

[23] Chollet, F. (2017). Xception: Deep learning with depthwise separable convolutions. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 1251-1258).DOI: 10.1109/CVPR.2017.195

[24] Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., & Wojna, Z. (2016). Rethinking the inception architecture for computer vision. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 2818-2826). DOI: 10.1109/CVPR.2016.308

[25] Howard, A., Sandler, M., Chu, G., Chen, L. C., Chen, B., Tan, M.,... & Adam, H. (2019). Searching for mobilenetv3. In *Proceedings of the IEEE/CVF international conference on computer vision* (pp. 1314-1324). DOI:10.1109/ICCV.2019.00140

[26] Tan, M., & Le, Q. (2021). Efficientnetv2: Smaller models and faster training. In *International conference on machine learning* (pp. 10096-10106). PMLR. arXiv:2104.00298

[27] Zoph, B., Vasudevan, V., Shlens, J., & Le, Q. V. (2018). Learning transferable architectures for scalable image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 8697-8705). arXiv:1707.07012

[28] Ho, Jonathan, Jain, Ajay, & Abbeel, Pieter (2020). Denoising Diffusion Probabilistic Models. In *NIPS'20: Proceedings of the 34th International Conference on Neural Information Processing Systems (574)* (pp. 6840 - 6851) arXiv:2006.11239

[29] Dosovitskiy, Alexey, et al. (2021). An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. In *International Conference on Learning Representations 2021*. arXiv:2010.11929v2

[30] Gulrajani, I., Ahmed, F., Arjovsky, M., Dumoulin, V., & Courville, A. C. (2017). Improved training of wasserstein gans. *Advances in neural information processing systems*, 30. arXiv:1704.00028

[31] Hasan, M., Khatun, S., Raihan, M. A., & Uddin, A. H. (2023). Rice Leaf Bacterial and Fungal Disease Dataset. *Mendeley Data*, V2. DOI: 10.17632/hx6f852hw4.2

[32] vbookshelf. (n.d.). Rice Leaf Diseases. Kaggle. Link

[33] Putra, D. I. W. (n.d.). Rice Leafs Disease Dataset. Kaggle. Link

[34] loki4514. (n.d.). Rice Leaf Diseases Detection. Kaggle. Link

[35] Anshul, M. (n.d.). Rice Disease Dataset. Kaggle. Link

[36] Riyaz, S. (n.d.). RiceLeafs. Kaggle. Link